Rendering Description (RD) Data File Format -- V2.2

This document describes the RD (rendering description) data file format. The format is meant to be a human readable format for the exchange of 3D data for scene modeling and rendering. The style and many commands are inspired partly by the RenderMan RIB format, partly by OpenGL. But there are some significant extensions, restrictions, and modifications.

Data Description

The commands in the RD format are all single words without spaces. The commands are case sensitive. Commands may or may not take arguments. Some commands, particularly the ones for point sets, line sets and polygon sets may have a variable number of arguments.

Command arguments may be boolean values, integers, floating point values and strings. Boolean values are represented by unquoted text strings. The logical value of true can be represented by any of the values `true`, `yes`, `on`, or `1`. The logical value of false can be represented by any of the values `false`, `no`, `off`, or `0`. Integers and floating point values follow the same conventions used for C/C++ style numeric constants. Strings in the file used as command arguments are enclosed in double quotation characters. Single quotes are not used. Strings may have spaces in them unless otherwise specified.

General Commands

Comments

The RD format supports comments in the data file. Comments are placed by using the `#` character. Everything following the character up through the end of the line is considered a comment.

`Display` *name type mode*

The `Display` command describes the type of device to which the image(s) will be rendered. The command takes 3 arguments which must be strings, each enclosed in quotes.

The *name* string may put a title on a window, or be used to name a file.

The *type* string describes which device the images will be rendered to. Each *type* string has one or more *mode* strings associated with it which describe further rendering options for the device. Not all modes are valid for all types. A list of registered type and mode combinations is given below.

The currently recognized display types are:

- `"Screen"` - Renders to the screen. The *name* argument is used as a title for the display window.

  This display type has three modes:

  - `"rgbsingle"` - color display in single buffered mode. The display is updated after each pixel.
  - `"rgbobject"` - color display in object buffered mode. The display is updated after each scene primitive.

    - `"rgbdouble"` - color display in double buffered mode. The display is updated after each frame, or at the end of the scene file.

- `"PNM"` - Renders to disk as a PBMPlus image file. The *name* argument is used to form a name for the image file. The frame number and a `.ppm` extension are added to the name before the file is written.

  This display type has only one mode:

    - `"rgb"` - output as a binary PPM file.

`Format` *width height*

This command takes two integer arguments specifying the size of the display in pixels. This command can only be used outside a world block (see below). Typically it is given shortly after a `Display` command. If this command is not given in a RD file, a default size of 640 pixels wide by 480 pixels high is used.

`Include` *filename*

This command takes a quoted string giving the filename of a file to include in the data stream. A filename of `"-"` will include input from the standard input stream. No check is made for recursive includes.

File Structuring Elements

Each RD file describes one or more images to be rendered to the output device. Multiple images are often used as the multiple frames of an animation, or as multiple views of a static scene to be used for image based modeling.

The commands for a particular frame constitute a *frame block*. Some commands must be given inside a frame block. Others must be given outside a frame block.

Each frame has associated with it a frame number. Frames are rendered in the order that they are encountered in the RD file. They are not rendered in order by frame number. This makes it possible to do out of order rendering to disk, which is useful for long animation sequences.

Within each frame, some rendering parameters can be altered before the rendering actually begins. These include camera placement, change of shading parameters, lighting and so forth. But when rendering begins, these parameters must be fixed. Depending on the style of renderer used, rendering can begin with the first object placed in the scene. (Other renderers may wait until all objects are placed in the scene before starting to render the image.) The concept of a *world block* is used to isolate object placement and description from the attributes which must be fixed when rendering begins.

Each frame block must contain one and only one world block. World blocks must not be nested.

A frame block is optional for rendering scenes consisting of one image. In such a scene the world block suffices to describe the frame. The frame block is assumed in this case, with a frame number of one (also assumed).

In an animation, it is common to have objects persist across multiple frames. In such a case, the repeated description of scence objects can take up a lot of file space and is prone to error if edited by hand. For this purpose the *object block* was created. An object block exists outside any frame block and is usually found at the beginning of a file. Multiple object blocks may exist, but they must not be nested. Each object block contains commands describing the placement of geometric primitives as well as possible changes of shading. Each object is assigned a name which is used to refer to the object.

Objects that have been defined in an object block can be used inside a world block or inside other object blocks through the use of object instances. Commands in an object block are not executed at the time the block is defined, but are executed at the time that the object is instanced. Object blocks are *not* recursive. An object instance for a particular object may not occur as part of the definition of that same object.

`FrameBegin` *frame_no*

The `FrameBegin` command declares the start of a frame block. The command takes one integer argument which is used as the frame number associated with the frame. The frame number is often ignored for images that are directed to the screen. For images sent to files, the frame number is often used to create a unique file name for the frame.

`FrameEnd`

The `FrameEnd` command is used to indicate the end of a frame block. Frame blocks may not be nested.

`WorldBegin`

The `WorldBegin` command starts a world block. It must occur either inside a frame block or stand alone if only one frame is to be rendered.

`WorldEnd`

This command ends a world block.

`ObjectBegin` *[n_params] object_name*

Starts the beginning of an (optionally) parameterized object block, which must be placed outside all frame and world blocks. Takes one or two arguments. The first, which is optional is a non-negative integer specifying the number of parameters that are supplied to the object. If not given, the number of parameters defaults to zero. The second parameter, a required string, must be in quotes, which is the name associated with the object. The only commands allowed inside an object block are geometry commands, geometric transformations and shading commands.

`ObjectEnd`

Indicates the end of an object block.

`ObjectInstance` *object_name [parameters]*

This command executes all of the commands inside the object block with the specified object name. `ObjectInstance` commands can occur inside world blocks or inside object blocks. The parameters given must match the definition of the object given in the corresponding `ObjectBegin` block.

Object Block Parameters

Within an object block, the parameters passed in are referred to by the parameter number, preceded by a dollar sign, i.e. `$1`, `$47`, etc. Parameters are single valued, but may be floating point values or strings. Parameters to an object block are numbered starting with 1. A parameter may be used (in theory), anywhere in the object block that its value would be acceptable.

Drawing Attributes

The RD format supports various ways of changing the appearance of objects and images. The most important of these have to do with color. Colors in the RD format are specified by three floating point numbers representing the amount of red, green, and blue respectively in the color. Colors are usually given in the range 0.0-1.0, giving a measure of device independence.

`Background` *r g b*

This command takes three floating point values in the range of 0.0-1.0 as a combination of red, green and blue to set the background color of the image. It makes no sense to change the background color during the middle of rendering an image. Consequently the `Background` command is allowed only outside a world block. If this command does not exist in the file, a default background color of black is used.

`Color` *r g b*

This command sets the *current drawing color* which will be used as the color of all successive objects until changed with another `Color` command. Valid color values are in the range of 0.0 to 1.0 for each component of the color. The default drawing color is white. The drawing color can be changed both inside and outside a world block.

`Opacity` *alpha*

This command sets the *current opacity* which will be used as the opacity of all successive objects until changed with another `Opacity` command. Valid opacities are in the range [0.0, 1.0], with 0.0 being fully transparent and 1.0 being fully opaque. The default opacity is 1.0. The opacity can be changed both inside and outside a world block.

Options

Options allow a scene file to set miscellaneous options associated with the rendering. Options are usually specific to a particular rendering engine. Each option has a name which is a quoted string as described above. The value associated with the option depends on its type, so several option types, listed below, have been provided. Some examples of each option are also provided, but there is no guarantee that these options exist for a given rendering engine.

`OptionArray` *name size values...*

This option allows a list of numbers to be provided. After the name of the option, an integer size is given, followed by the list of values.

`OptionBool` *name value*

The value is a boolean value, as described above.

Some common boolean options are listed below.

`Interpolate`

This option determines whether interpolated values are used in lighting calculations. The most important values that are interpolated are vertex normals and vertex colors. The default value of the interpolation flag is `on`.

`Wireframe`

This option determines whether primitives are rendered as solid surfaces or as wireframe objects. The default value of the wireframe flag is `off`.

`OptionList` *name size values...*

This option allows a list of quoted strings to be provided as values. After the name of the option, the integer number of values is given, then the quoted string values themselves.

`OptionReal` *name value*

The value of this option is a single numeric value. Some possible numeric options are

`Sides`

Determines the number of polygonal sections into which curved objects are broken down.

`OptionString` *name value*

The value of this option is a single quoted string as described above.

Camera Specification

Camera commands specify a view of the scene being created. They must occur outside a world block.

`CameraAt` *x y z*

Specifies the point in world coordinates that the camera is looking at. This point will appear in the center of the image. The default value is the point (0, 0, -1), so that combined with the default `CameraEye` below, the camera is looking down the negative z axis.

`CameraEye` *x y z*

Specifies the point in world coordinates that the camera is looking from. This point is also known as the *eyepoint*. The default value is the point (0, 0, 0).

`CameraFOV` *angle*

This command sets the field of view of the camera. It is specified in degrees and represents the vertical field of view. The default field of view is 90 degrees, which is unnaturally large. Small fields of view act like a zoom lens. Large fields of view act like a wide angle lens and increase the distortion in the image. Typical fields of view range from 20 to 60 degrees.

`CameraUp` *x y z*

Specifies a vector in world coordinates that points in a direction that will be seen as straight up in the image. The default value is the vector (0, 1, 0), i.e. the y axis is up. The default values of `CameraEye`, `CameraAt`, and `CameraUp` place the x and y world axes on the screen with the origin at the screen center and the axes in the traditional 2D Cartesian orientation.

`Clipping` *near far*

This command is followed by the depths of the near and far clipping planes. Depths are distances from the eye in camera coordinates. Default values are 1.0 for the near clipping plane and 1.0e+09 for the far clipping plane. As a general rule of thumb, clipping plane values should bound the scene as tightly as possible. This improves the effectvie resolution of any depth buffers used and the final quality of the image.

Geometry Commands

All geometry commands must be placed inside a world block or an object block.

Vertex Types

Some of the commands below dealing with sets of vertices allow the specification of a vertex type. The vertex type allows different attributes to be specified for an object on a per vertex basis. The vertex type is given as a quoted string consisting of characters from a limited set of attribute specifiers. All vertex attributes except the position (`"P"`) are optional.

The known vertex type attributes are:

`P`
: Position. Has three floating point components, x, y, and z.

`D` or `N`
: Direction vector. Has three floating point components, x, y, and z. For two dimensional primitives, i.e., surfaces, this is the surface normal vector. For one dimensional primitives, i.e., `LineSet`, and `Curve`, this component represents the tangent vector of the primitive.

`C`
: Surface color. Has three floating point components, r, g, and b representing the amount of red, green, and blue respectively. The components should have values between 0.0 and 1.0 inclusive.

`w`
: Weight. Has one floating point value. Used to provide weight values for `Curves` and `Patches`. Almost always found in conjunction with vertex type `P`.

`T`
: Texture. For surfaces, the texture is represented as two floating point components, s and t, which are the horizontal and vertical components of the texture. For curves, the texture is a single floating point component.

`O`
: Opacity. A single floating point value with 0.0 being totally transparent and 1.0 being totally opaque.

Angular Extents

Some of the solid objects below allow angular extents. This is primarily for objects that are produced by sweeps around an axis. All angular extents are in degrees and proceed clockwise around the z axis. Unless otherwise specified, all extents start at the positive x axis. For example, an angular extent of 180 would produce half of the swept object; an angular extent of 360 degrees, the entire object.

`Point` *x y z*

This command is followed by the x, y, and z coordinates of the point. The color of the point is determined by the current drawing color (See the section on drawing attributes).

`PointSet`

Followed by a vertex type, and the number of points in the set. The listed attributes follow for each point in the order specified.

Example

```
PointSet "PC"
3 # particles - x y z  r g b
 3.5 4.5  2.0  0.0 0.5 0.5 # medium cyan
-1.0 2.5 -3.4  1.0 0.5 0.0 # orange
 0.0 1.0  1.0  0.0 0.0 0.0 # black
```

`Line` x_1 y_1 z_1 x_2 y_2 z_2

Followed by the x, y, z of the first endpoint and the x, y, z of the second endpoint. The color of the line is determined by the current drawing color.

`LineSet`

Followed by a vertex type, then the number of vertices and line sequences (polylines) in the object. The given attributes follow for each vertex in the order specified in the vertex type. After the vertex list, a list of polylines follows. Each polyline is given as a list of indices into the list of attributed vertices. The list of vertices for a given polyline is followed by the value -1 to indicate the end of the vertices for that polyline. The number of vertices per polyline is arbitrary and may change from polyline to polyline.

Example 1

```
LineSet "PC"
8 6 # Wireframe color cube - x y z  r g b
 -1.0 -1.0 -1.0 0.0 0.0 0.0  # Black
  1.0 -1.0 -1.0 1.0 0.0 0.0  # Red
 -1.0  1.0 -1.0 0.0 1.0 0.0  # Green
  1.0  1.0 -1.0 1.0 1.0 0.0  # Yellow
 -1.0 -1.0  1.0 0.0 0.0 1.0  # Blue
  1.0 -1.0  1.0 1.0 0.0 1.0  # Magenta
 -1.0  1.0  1.0 0.0 1.0 1.0  # Cyan
  1.0  1.0  1.0 1.0 1.0 1.0  # White

0 1 5 7 -1 # Black Red Magenta White
0 2 3 7 -1 # Black Green Yellow White
0 4 6 7 -1 # Black Blue Cyan White
1 3 -1     # Red Yelow
2 6 -1     # Green Cyan
3 5 -1     # Blue Magenta
```

Example 2

```
LineSet "PC"
8 6 # Wireframe color cube - x y z  r g b
 -1.0 -1.0 -1.0 0.0 0.0 0.0  # Black
  1.0 -1.0 -1.0 1.0 0.0 0.0  # Red
 -1.0  1.0 -1.0 0.0 1.0 0.0  # Green
  1.0  1.0 -1.0 1.0 1.0 0.0  # Yellow
 -1.0 -1.0  1.0 0.0 0.0 1.0  # Blue
  1.0 -1.0  1.0 1.0 0.0 1.0  # Magenta
 -1.0  1.0  1.0 0.0 1.0 1.0  # Cyan
  1.0  1.0  1.0 1.0 1.0 1.0  # White
```

```
0 2 3 1 -1 # Red Green Yellow face
0 1 5 4 -1 # Red Blue Magenta face
1 3 7 5 -1 # Red Yellow Magenta face
3 2 6 7 -1 # Yellow Green Cyan face
2 0 4 6 -1 # Green Blue Cyan face
4 5 7 6 -1 # Blue Magenta Cyan face
```

`Circle` *x y z radius*

Draws a circle on the screen. The `Circle` command is followed by the x, y, and z coordinates of the circle center and the radius of the circle. The circle is always drawn parallel to the screen and will thus always appear as a circle.

`Fill` *x y z*

The `Fill` command performs a 4-connected flood fill on the screen starting at the projection of the given seed point. If the seed point is not on the screen, no fill is done. The current drawing color is used as the fill color.

`Cone` *height radius theta*

Renders a cone with its base in the xy plane, centered at the origin. The height is in the positive z direction. The angle theta is the angular extent mentioned above.

`Cube`

Takes no arguments. Renders an axis aligned cube extending from -1 to +1 in x, y, and z.

`Curve`

Followed by the curve type in quotes and the vertex type. Remaining arguments depend on the type of the curve. Current curve types include

`Bezier`
: After the vertex type is the degree of the curve. The control point vertices follow. If the degree of the curve is n, there should be exactly n+1 control points.

Example

```
Curve "Bezier" "PC"
3 # cubic curve with color - x y z  r g b
-0.5 0.0 -1.0 1.0 0.0 0.0  # Red
-0.5 0.5 -1.0 1.0 1.0 0.0  # Yellow
0.5 -0.5 -1.0 1.0 1.0 1.0  # White
0.5  0.0 -1.0 0.0 1.0 0.0  # Green
```

`Cylinder` *radius zmin zmax theta*

Produces a cylinder centered around the z axis with the given radius, extending from zmin to zmax along the axis. The zmin value should be less than the zmax value. Theta is the angular extent of the cylinder, as described above.

`Disk` *height radius theta*

Produces a disk of the given radius parallel to the xy plane and centered around the z axis at the given height. Theta is the angular extent of the disk, as described above. The surface normal of the disk is outward in the positive z direction.

`Hyperboloid` *x_1 y_1 z_1 x_2 y_2 z_2 theta*

Produces a hyperboloid of one sheet centered around the z axis by sweeping a line with the given endpoints around the z axis. Theta gives the angular extent of the sweep, as described above.

`Paraboloid` *radius zmin zmax theta*

Produces a paraboloid of the given radius and extents in z centered about the z axis. Theta is the angular extent, described above.

`Patch`

Followed by the patch type in quotes and the vertex type. Remaining arguments depend on the type of the patch. Current patch types include

`Bezier`
: After the vertex type are two integers, the degrees of the patch in the u and v directions respectively. The control point vertices follow. If the degree of the patch is m by n, there should be exactly (m+1)(n+1) control points.

Example

```
Patch "Bezier" "PC"
2 2 # biquadratic patch with color - x y z  r g b
-0.5 -0.5 -1.0 1.0 0.0 0.0  # Red
 0.0 -0.5 -0.5 1.0 1.0 0.0  # Yellow
 0.5 -0.5 -1.0 1.0 1.0 1.0  # White

-0.7  0.0  0.0 1.0 0.0 1.0  # Magenta
 0.0  0.2 -1.0 0.5 0.5 0.5  # Gray
 0.7  0.0  0.0 0.0 1.0 0.0  # Green

-0.5  0.5  0.0 1.0 1.0 1.0  # White
 0.1  0.7  1.0 1.0 1.0 0.0  # Yellow
 0.5  0.5 -1.0 0.0 1.0 1.0  # Cyan
```

`PolySet`

Followed by a vertex type, then the number of vertices and faces in the object. The given attributes follow for each vertex in the order specified in the vertex type. After the vertex list, a list of faces follows. Each face is given as a list of indices into the list of attributed vertices. The list of vertices for a given face is followed by the value -1 to indicate the end of the vertices for that face. The number of vertices per face is arbitrary and may change from face to face. The vertices of any given face should be traced in a counter-clockwise order as seen from the exterior of the object.

Example

```
PolySet "PC"
8 6 # Color cube - x y z  r g b
 -1.0 -1.0 -1.0 0.0 0.0 0.0  # Black
```

```
  1.0 -1.0 -1.0 1.0 0.0 0.0  # Red
 -1.0  1.0 -1.0 0.0 1.0 0.0  # Green
  1.0  1.0 -1.0 1.0 1.0 0.0  # Yellow
 -1.0 -1.0  1.0 0.0 0.0 1.0  # Blue
  1.0 -1.0  1.0 1.0 0.0 1.0  # Magenta
 -1.0  1.0  1.0 0.0 1.0 1.0  # Cyan
  1.0  1.0  1.0 1.0 1.0 1.0  # White

0 2 3 1 -1 # Red Green Yellow face
0 1 5 4 -1 # Red Blue Magenta face
1 3 7 5 -1 # Red Yellow Magenta face
3 2 6 7 -1 # Yellow Green Cyan face
2 0 4 6 -1 # Green Blue Cyan face
4 5 7 6 -1 # Blue Magenta Cyan face
```

`Sphere` *radius zmin zmax theta*

Renders a spherical section centered about the origin using the given radius and extents. The value of zmin should be less than zmax. If zmin and zmax are outside the radius of the sphere, they are effectively replaced with -radius and radius, respectively.

`SqSphere` *radius n e zmin zmax theta*

Renders a superquadric spherical section centered about the origin using the given radius, north and east parameters, and extents. The north parameter is the power used on the latitude; the east, on the longitude. Values of 1.0 produce circular cross sections. Values of 0.0 produce square cross sections. Values of 2.0 produce diamond cross sections. The value of zmin should be less than zmax. If zmin and zmax are outside the radius of the superquadric sphere, they are effectively replaced with -radius and radius, respectively.

`SqTorus` *radius1 radius2 n e phimin phimax thetamax*

Renders a superquadric torus (doughnut shape) parallel to the xy plane centered about the origin. Radius1 is the radius of the major circle shape of the torus. It is the radius of the center of the toroidal cross section. Radius2 is the radius of the small circle. n and e are the values of the exponents used in the superquadric sin and cos formulas. Values of 0.0 produce square cross sections. Values of 1.0 produce circular cross sections. Values of 2.0 produce diamond cross sections. Phi varies between phimin and phimax. Theta varies between 0 and thetamax. The bounds on phi and theta represent geometrical bounds and must likely be converted to parameter bounds.

`Subdivision`

Followed by the subdivision surface type in quotes and the vertex type. Remaining arguments depend on the type of subdivision surface. Current subdivision surface types include

`Catmull-Clark`
: After the vertex type are three integers, the number of vertices, number of faces, and number of creases in the surface. The control point vertices follow. These are followed by the list of vertex indices for each face, listed counter-clockwise as seen from the outside with each face vertex list terminated by a -1, exactly as used in `PolySet`. Each crease list is followed immediately by a equally sized list of sharpness values for the crease.

`Torus` *radius1 radius2 phimin phimax thetamax*

Renders a torus (doughnut shape) parallel to the xy plane centered about the origin. Radius1 is the radius of the major circle shape of the torus. It is the radius of the center of the toroidal cross section. Radius2 is the radius of

the small circle. Phi varies between phimin and phimax. Theta varies between 0 and thetamax. Unfortunately, the best way to describe this one is with a picture or formulas:

- For a given theta and phi,
- r = radius2 * cos(phi)
- z = radius2 * sin(phi)
- x = (radius1 + r) * cos(theta)
- y = (radius1 + r) * sin(theta)

`Tube` x_1 y_1 z_1 x_2 y_2 z_2 *radius*

Produces a cylinder of given radius between the two given points. This is strictly a convenience primitive. There is nothing here that can't be done with a `Cylinder` and the proper transformations.

Geometric Transformations

All points are passed through a geometric transformation pipeline. Various transformations can be pushed on the front of the pipeline. The order in which various transformations occur is very important. Because the transformations are placed on the front of the transformation pipeline, they appear in the scene file in reverse order, i.e. later transformations in the pipeline occur earlier in the file.

`Matrix` *16 values*

This command is followed by 16 floating point values which represent a 4x4 transformation matrix. The values are listed in row major order. Translation values are placed in the last column. This matrix is pushed on the front of the geometric transformation pipeline. It does **not** replace the current value of the pipeline.

This command is intended for use in rare situations. It is almost always better to use the Rotate, Scale, and Translate commands listed below.

`Rotate` *axis* *angle*

This command is followed by a principal axis, in quotes, around which the rotation occurs. Valid values are `"X"`, `"Y"`, and `"Z"`. Positive rotation around the axis is seen as counter clockwise around the axis when looking down the positive axis towards the origin. The angle of the rotation is given in degrees.

`Scale` s_x s_y s_z

This command scales the x, y, and z coordinates of points by the values given for s_x, s_y, and s_z respectively.

`Translate` t_x t_y t_z

Translates points by the given translation values in x, y, and z.

`XformPush`

Pushes the current transformation from object coordinates to world coordinates onto a stack. This preserves the current transformation state, which may be changed by subsequent geometric transformations. The current transformation is implicitly pushed upon the stack before every `ObjectInstance` command.

`XformPop`

Pops a transformation from the transformation stack and places it as the current transformation from object to world coordinates. The current transformation is implicitly popped off the stack after every `ObjectInstance` command.

Lighting

The lighting commands allow lights to be placed in the scene which affect the appearance of scene objects. There are currently three types of lights, an ambient light, distant lights, and point light sources. Each light has a color and an intensity. Internally, each color component is multiplied by the intensity to obtain a color component intensity which is used in the lighting calculations. In the RD format file, the color and intensity are kept separate for ease in manipulating light intensity while leaving light color constant.

Currently there is no saved lighting state. Any light placed in the scene description is used from that point on in the lighting calculations. All lighting information is lost at the end of a frame block or world block. Lights can not be set outside a world block or frame block. This implies that lights are not cumulative between the frames of an animation, but they must be set in each frame. Lights can not be turned off or removed from the scene.

`AmbientLight` r g b I

Specifies an ambient light, which comes from everywhere in a scene. The parameters are the color of the light and its intensity. The color values are in the range 0.0-1.0 and the intensity can be any non-negative value. Used primarily to keep shadow areas from becoming too dark. There can only be one ambient light in the scene. The default ambient light is white with an intensity of 1.0.

`FarLight` l_x l_y l_z r g b I

This command describes a distant light source, one whose intensity does not fall off with distance. The first three command arguments describe the x, y, and z components of the light vector, L, the direction the light shines. The remaining arguments describe the color and intensity of the light.

There may be multiple `FarLights` in a scene. The maximum number is not specified but is implementation dependent. The default number of `FarLights` in a scene is zero.

`PointLight` p_x p_y p_z r g b I

This command describes a point light source, one whose intensity falls off with the square of the distance. The first three command arguments describe the x, y, and z coordinates of the light position. The remaining arguments describe the color and intensity of the light.

There may be multiple `PointLights` in a scene. The maximum number is not specified but is implementation dependent. The default number of `PointLights` in a scene is zero.

`ConeLight` p_x p_y p_z a_x a_y a_z $theta_{min}$ $theta_{max}$ r g b I

This command describes a cone light source, one whose intensity falls off with the square of the distance. The first three command arguments describe the x, y, and z coordinates of the light position. The next three arguments describe a point that the light is directed at.

The next two arguments describe the shape of the light. The angles (in degrees) describe the full angular extent of two cones with apices at the source and centered about the viewing direction. The intensity of the light is 0 outside the cone described by $theta_{max}$, and gradually transitions to full intensity inside the inner cone. The details of the transition between no intensity and full intensity are implementation dependent.

The remaining arguments describe the color and intensity of the light.

Surface Attributes

Objects in the scene can be assigned various surface attributes which affect the appearance of the object. The most important of these is the surface color, which is set through the `Color` command and is discussed in the Drawing Attributes section. Other attributes are used to calculate how the lights described in the Lighting section above interact with the object.

An extremely important surface attributes is the surface shader, a function which calculates the final color to be returned to the output device based on the other surface attributes and lights. There are several built-in shaders. Not all shaders use all of the other attributes used to describe the surface.

Currently there is no saved surface attribute state. Any attributes changed inside an `ObjectInstance` remain in effect upon return from the instance. Attribute values hold between frames of an animation as well. This implies that surface attributes can be set outside a frame block or world block.

`Ka`

The ambient coefficient of the surface. Has a value in the range 0.0-1.0. Describes the fraction of the ambient light returned by the surface. The default value is 1.0.

`Kd`

The diffuse coefficient of the surface. Has a value in the range 0.0-1.0. Describes the fraction of the diffuse light returned by the surface. The default value is 1.0. A good heuristic for this value is to set it equal to `Ka`.

`Ks`

The specular coefficient of the surface. Has a value in the range 0.0-1.0. Describes the fraction of the diffuse light returned by the surface. The default value is 0.0. A good heuristic for this value is to set it equal to `1.0 - Kd`.

`Specular` *r g b* n_s

Gives the specular color and specular exponent of the object. The color components are in the range 0.0-1.0. The specular exponent can be any positive non-zero value. In the real world, the color highlights of an object may be completely independent of the base color of the object. To simulate many plastics, this color should be white. To simulate metal, this color should be the same as the base color of the object.

The specular exponent determines the size of the highlights. A large exponent produces small, tight highlights. A small exponent produces large, diffuse highlights.

The default specular color is white (1.0 1.0 1.0) and the default specular exponent is 10.

`Surface` *shader*

This command sets the surface shading function used to render all subsequent objects. The shader attribute is a quote enclosed string. Valid shader values include `"matte"`, `"metal"`, and `"plastic"`. These shaders are described below. The default surface shader is `"matte"`. In the formulas below, C_s is the surface color; L_a is the ambient light intensity; I_d is the reflected diffuse intensity, obtained from the surface normal and the `FarLight`

intensities and directions and the `PointLight` intensities and positions; C_{spec} the specular surface color; I_s is the specular reflection intensity determined by the surface normal, viewing vector, specular exponent, and the `FarLight` and `PointLight` intensities and geometries.

`"matte"`
: Produces diffuse or matte effects. There is no specular contribution in this shader. The formula for computing the sample pixel intensity is
C_s (`Ka` * L_a + `Kd` * I_d)

`"metal"`
: Produces metallic reflection effects. There is no diffuse reflection contribution in this shader. The formula for computing the sample pixel intensity is
C_s (`Ka` * L_a + `Ks` * I_s)

`"plastic"`
: Produces general diffuse and specular reflection effects. With the right parameters, can simulate both `matte` and `metal` shaders. The formula for computing the sample pixel intensity is
C_s (`Ka` * L_a + `Kd` * I_d) + C_{spec} * `Ks` * I_s

`"painted plastic"`
: Produces general diffuse and specular reflection effects. Uses texture mapping. Identical to the `plastic` surface shader except that the surface color C_s comes from a texture map. The formula for computing the sample pixel intensity is
$C_{s_{text}}$ (`Ka` * L_a + `Kd` * I_d) + C_{spec} * `Ks` * I_s
The specular surface color is not adjustable by a texture map with this shader.

Attribute Mapping

Various parameters in the pixel shading calculation can be altered through the use of *attribute maps*. Attribute maps allow shading paramteters to be changed frequently between object vertices. This can give the appearance of a great deal of model complexity where in fact, the geometrical complexity may be small.

The most common type of attribute map is a *texture map* which changes the surface color. Texture maps have the appearance of pasting an image on the surface. Another common map is the *bump map* which alters the surface normal used in lighting calculations. A bump map makes the surface appear bumpy.

Many of the commands below take a string indicating map type. Map type values are reserved and can *not* be used as map labels. Valid map type values include `"none"`, `"texture"`, and `"bump"`.

`MapLoad` *filename label*

This command takes two string arguments. Loads a PBMplus image file into memory to be used as an attribute map. A hierarchical mipmap is created of the map. The label is used to refer to the texture in later rendering. The label `"none"` is reserved and may not be assigned as a label.

`Map` *type label*

Takes two string arguments. Makes the map referred to by the label the current map used for rendering objects. Only one map of a given type is considered active at a time. If the label `"none"` is used, then no mapping is performed for that attribute type.

`MapSample` *type intra_level inter_level*

This command takes three string arguments which determine the type of sampling used in mapping. The first arugment determines the map type that will be affected by these sampling commands. The second argument determines how sampling is performed between pixels on the same level of a mipmap. The third argument determines how sampling is performed between levels of a mipmap. The allowable values for the intra_level and inter_level arguments are

`"nearest"`
: - nearest neighbor sampling.

`"linear"`
: - linear interpolation

The combination `"texture" "linear" "nearest"` represents standard bilinear texture mapping. The combination `"texture" "linear" "linear"` represents standard trilinear texture mapping.

MapBound *type* s_{min} t_{min} s_{max} t_{max}

Takes a string and four floating point arguments. This command is used to help set map coordinate bounds on primitives where map coordinates are not explicitly specified. These include spheres, cones, and cylinders. By default the map coordinate bounds are from 0 to 1 in s and t. The bounds set remain in force until explicitly reset.

MapBorder *type* *horizontal* *vertical*

Takes three string arguments. This command determines how maps are sampled at the borders. This is especially noticeable when interpolation is done at the edges. Allowed values are

`"none"`
: treats values outside the map as having a constant value. For texture maps this is the same as having the current surface color. This is useful for a gradual texture edge falloff at the edge of a polygon.

`"clamp"`
: values outside the map are clamped to the value of the closest edge. This is best for adjacent tiles.

`"repeat"`
: considers the map to be a periodically repeating tile. Values that leave one edge of the map are considered to start again on the opposite side. This is best for maps that wrap all the way around an object.

The method used to handle the borders is independent in both the horizontal and vertical map directions.